CSCI 12532
Computer Architecture and Design
Lecture 06

Ms. Meesha Mervyn
Dept. of Computer Systems Engineering
Faculty of Computing and Technology
University of Kelaniya
meesham@kln.ac.lk

Contents

1. INSTRUCTION PIPELINING

A CPU typically processes an instruction through four stages.

- Fetch: Fetch the instruction from memory.
- Decode: Decode the instruction to understand what operation is required.
- Execute: Execute the operation (e.g., arithmetic or logic operations).
- Write: Write the result back to memory or a register.

Assume that each of these stages takes one cycle to complete. Therefore, processing a single instruction would take four cycles. And processing two instructions would then take eight cycles.

If we were a CPU designer, how could we make things faster?

To solve this inefficiency, they introduced the concept of instruction pipelining.

	FU	DU	EU	WU
t0 + 1	i1			
t0 + 2	i2	i1		
t0 + 3	i3	i2	i1	
t0 + 4	i4	i3	i2	i1
t0 + 5	i5	i4	i3	i2
t0 + 6	i6	i5	i4	i3
t0 + 7	i7	i6	i5	i4
t0 + 8	i8	i7	i6	i5

...

What is Instruction Pipelining?

This technique is used in CPUs to improve performance by overlapping the execution of multiple instructions.

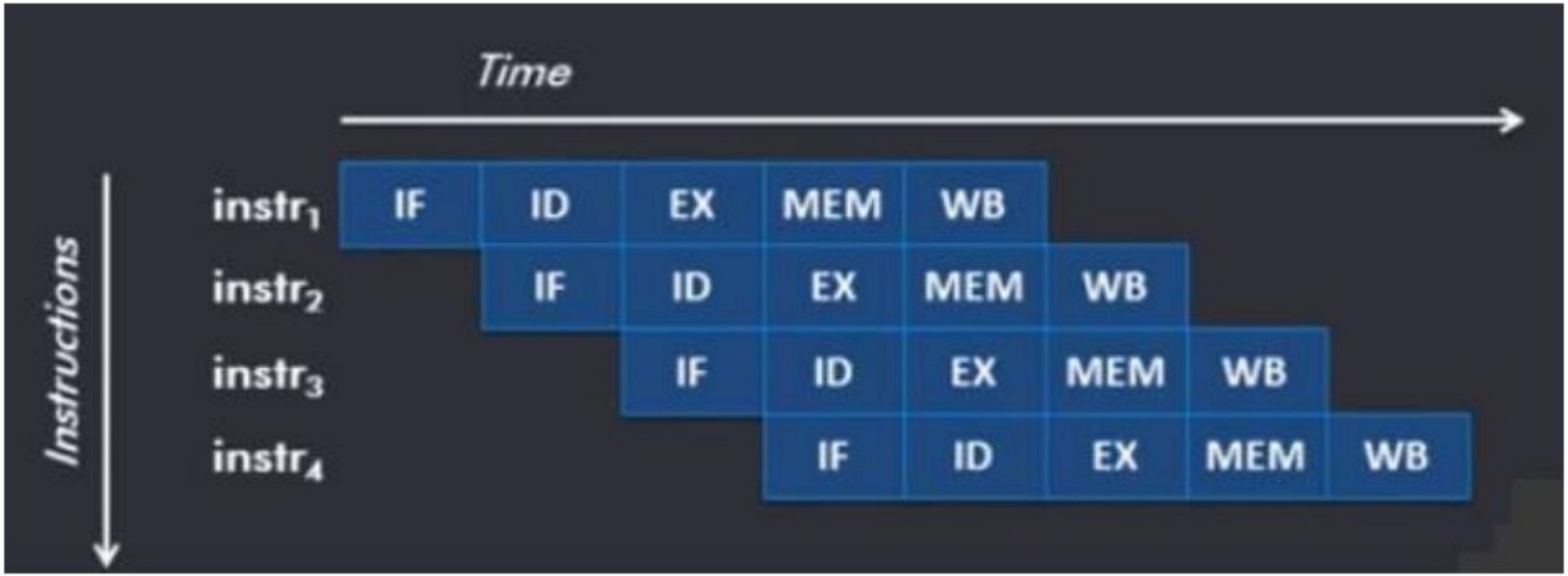
The pipeline organization of a CPU is similar to an assembly line: the work to be done in an instruction is broken into smaller steps (pieces), each of which takes a fraction of the time needed to complete the entire instruction. Each of these steps is a pipe stage (or a pipe segment).

This improves the throughput (more instructions completed per cycle).

Canonical 5 Stage Pipeline in MIPS

1. Instruction Fetch (IF) – Get instruction from memory.
2. Instruction Decode (ID) – Decode instruction & read registers.
3. Execute (EX) – Perform operation (e.g., ALU calculation).
4. Memory Access (MEM) – Read/write data memory (if needed).
5. Write Back (WB) – Store result back to a register.

Pipelined Execution means each stage runs in parallel for different instructions. While Instruction 1 is in EX, Instruction 2 is in ID, and Instruction 3 is in IF and so on... Speed-Up the execution. Ideally 5x faster (for many instructions) since 5 stages work in parallel.


Time
Instructions
instr1
instr2
instr3
instr4
IF
ID
EX
MEM
WB

8086 Pipeline

The Intel 8086 is a 16-bit microprocessor that laid the foundation for modern processors. It follows the Von Neumann architecture, where data and instructions share the same memory.
The 8086 microprocessor introduced powerful features like segmentation, pipelining, and multitasking support, making it a stepping stone for modern CPUs.

8086 Pipeline Overlaps Fetch & Execute:
While the n^{th} instruction is being executed, the $(n+1)^{th}$ instruction is being fetched.
Increases processing speed by reducing idle time between instructions.

Acceleration by Pipelining

- τ: duration of one cycle
- n: number of instructions to execute
- k: number of pipeline stages
- Tk,n: total time to execute n instructions on a pipeline with k stages
- Sk,n: (theoretical) speedup produced by a pipeline with k stages when
- executing n instructions

$$T_{k,n} = [k + (n-1)] \times \tau$$

- The first instruction takes k*τ to finish
- The following n − 1 instructions produce one result per cycle.

On a non-pipelined processor each instruction takes k*τ, and n instructions take Tn = n*k*τ

$$S_{k,n} = \frac{T_n}{T_{k,n}} = \frac{n \times k \times \tau}{[k + (n-1)] \times \tau} = \frac{n \times k}{k + (n-1)}$$

For large number of instructions (n → ∞) the speedup approaches k (no. of stages).

Apparently a greater number of stages always provides better performance.

However:

- A greater number of stages increases the overhead in moving information between stages and synchronization between stages.
- With the number of stages the complexity of the CPU grows.
- It is difficult to keep a large pipeline at maximum rate because of pipeline hazards.

Pipelining Explained using Laundry Example

Non-Pipelined Approach is as follows.

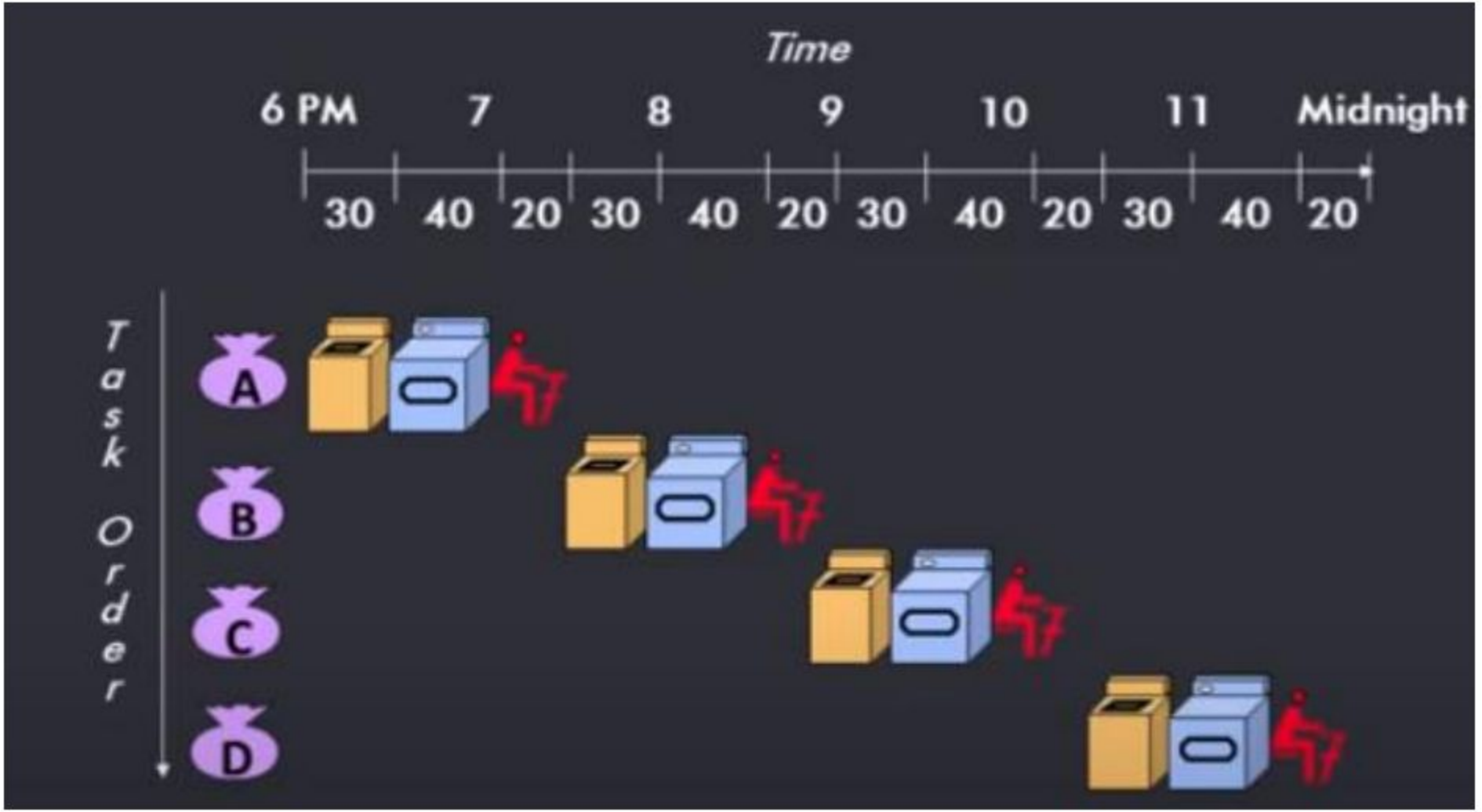
- Step 1: Put one dirty load in the washer.
- Step 2: When washing is done, move wet load to the dryer.
- Step 3: When drying is done, move dry load to fold.
- Step 4: When folding is done, have roommate put clothes away.
- Repeat for the next load.

Problem: Slow, only one task at a time.

Pipelined Approach will be as follows.

- Overlap tasks by starting the next load as soon as a stage is free.
  Example:
  - Washer finishes Load 1 ➔ move to dryer, start Load 2 in washer.
  - Dryer finishes Load 1 ➔ move to folding, move Load 2 to dryer, start Load 3 in washer.
  - Continue until all stages (washing, drying, folding, putting away) run in parallel.

Faster for multiple loads because tasks happen simultaneously.


Time
6 PM
7
8
9
10
11
Midnight
30
40
20
30
40
20
30
40
20
30
40
20
Task Order
A
B
C
D

Does NOT reduce time for a single load (still takes the same time).
Improves throughput (more loads finished per hour).
Speed-up potential = number of stages (e.g., 4 stages ➔ up to 4x faster).
Best when:

- All stages take similar time. (Uniform instruction length)
- Many loads (pipeline stays full).

Less efficient for few loads (start-up/wind-down time reduces gains).

How the speed-up works?
Non-pipelined: 20 loads take 20x longer than 1 load.
Pipelined (4 stages): 20 loads take ~5x longer than 1 load (close to 4x speed-up).
With only 4 loads: Speed-up is less (e.g., 2.3x) because pipeline isn't always full.

What Makes Pipelining Easy in MIPS?

1. Fixed-Length Instructions (All MIPS Instructions Same Length)
   - Every MIPS instruction is 32 bits long. The CPU can always fetch instructions in a predictable way (no need to check instruction size).

2. Few and Regular Instruction Formats
   - MIPS has only three main instruction formats: R-type (Register), I-type (Immediate), J-type (Jump)
   - Decoding (ID stage) is simpler because fields (opcode, registers, immediates) are always in the same positions.

3. Only Load and Store Instructions Access Memory
   - Most MIPS instructions operate only on registers. Only lw (load word) and sw (store word) interact with memory.
   - Memory Access (MEM stage) is needed only for these two instructions. Most instructions skip MEM, reducing pipeline stalls.

2. PIPELINING HAZARDS

There are situations in pipelining when the next instruction cannot execute in the following clock cycle. These events are called hazards, and there are three different types.

- Structural Hazards
- Data Hazards
- Control Hazards

Structural Hazards

A structural hazard occurs when there is a resource conflict in the hardware. The hardware simply cannot support all possible combinations of instructions executing simultaneously.
In a non-Harvard architecture (where instructions and data share a single unified cache), a structural hazard occurs if an instruction in the Instruction Fetch (IF) stage attempts to read the cache at the same time an earlier instruction in the Memory (MEM) stage attempts to read or write data to that same cache.

Resolving Structural Hazards

1. Resource Duplication: Adding extra hardware—such as separate instruction and data caches, or extra ALUs—allows simultaneous access.
2. Out-of-Order Execution: Instructions are dispatched out of order when resources are free, reducing resource bottlenecks.

Data Hazards

A data hazard is a data access-related conflict that will produce an incorrect value if unresolved. The absolute precondition for a data hazard is a data dependency. A data dependency occurs when two or more instructions need to act on a common operand. Dependencies are an unavoidable feature of programming, and there are several types.

1. True Data Dependency: This occurs when an instruction consumes a data item that was produced by a prior instruction.
2. False / Name Dependencies: These occur when two instructions use the same register name, but no actual data flows between them. Because we only have a limited number of registers (e.g., 32 in RISC-V), programmers and compilers are forced to reuse them, creating these pseudo-dependencies.

Data Hazard Types

1. RAW (Read After Write): Based on a True Data Dependency. The programmer intended for a read to take place after a write. A RAW hazard occurs if the pipeline overlaps execution such that the read accidentally happens before the write is finished. In our standard RISC-V pipeline, this is the most common hazard and the primary reason we must stall.
2. WAW (Write After Write): Based on an Output Dependency. The programmer intended for one write to happen after another write. A WAW hazard occurs if the pipeline jumbles the order and the second write happens before the first, leaving the wrong final value in the register. Simple pipelines do not suffer from WAW hazards, but advanced pipelines (like varying latency or dynamically scheduled pipelines) do.
3. WAR (Write After Read): Based on an Anti-Dependency. The programmer intended for a write to happen after a read. A WAR hazard occurs if the pipeline executes out of order and allows the write to happen before the read takes place, causing the reader to ingest the new (wrong) value. Our standard pipeline naturally avoids WAR hazards because reads always happen early (in ID) and writes always happen late (in WB).

Resolving Data Hazards

1. Stalling (The Bulletproof Method)

The most reliable, foolproof method to resolve a hazard is to simply stall the pipeline. The ID stage does the heavy lifting: it decodes the operands, checks all currently executing instructions for dependencies, and if it detects that an instruction will not produce its result in time, it forces the current instruction to recycle in the ID stage.

2. Forwarding Through the Register File

In the standard pipeline, an instruction writing to the register file (in the WB stage) and an instruction reading from it (in the ID stage) might collide. To resolve this without a stall, the pipeline modifies the hardware timing: the WB stage is programmed to write to the register file on the rising edge (the first half) of the clock pulse, and the ID stage reads on the falling edge (the second half).

Control Hazards / Branch Hazards

A control hazard involves a transfer-of-control conflict that disrupts the normal sequential flow of the program. The precondition is a control dependency, which dictates that any instruction immediately following a branch (the fall-through) or any instruction that is the target of a branch is control-dependent on that branch. Because the pipeline evaluates the branch condition and updates the Program Counter later in the pipeline (often in the EX or MEM stage), the pipeline will have already fetched the wrong instructions, necessitating a stall or a flush (converting the incorrectly fetched instructions into NOOPs)

Resolving Control Hazards

1. Branch Prediction: Hardware predicts whether a branch will be taken or not and fetches instructions accordingly.
2. Branch Target Buffers (BTB): Cache memory holds the target addresses of previously executed branches to speed up fetching.
3. Delayed Branch: The compiler rearranges the sequence to execute useful instructions in the "branch delay slot" while the condition is evaluated.
4. Pipeline Flushing: If a prediction is incorrect, the incorrectly fetched instructions are discarded or flushed from the pipeline.

3. ADVANTAGES AND DISADVANTAGES O PIPELINING

Advantages of Pipelining

- Instruction throughput increases.
- Increase in the number of pipeline stages increases the number of instructions executed simultaneously.
- Faster ALU can be designed when pipelining is used.
- Pipelined CPU’s works at higher clock frequencies than the RAM.
- Pipelining increases the overall performance of the CPU.

Disadvantages of Pipelining

- Designing of the pipelined processor is complex.
- Instruction latency increases in pipelined processors.
- The throughput of a pipelined processor is difficult to predict.
- The longer the pipeline, worse the problem of hazard for branch instructions.

Thank You

19.05.2026